

INFO0503

Autour du HTTP

INTRODUCTION À LA PROGRAMMATION CLIENT / SERVEUR

L3 INFORMATIQUE

OLIVIER FLAUZAC & CYRIL RABAT



Plan

HTTP et éléments Web

Le protocole HTTP

Echange de données : les formulaires HTML

Sessions et Cookies

PHP et HTTP

- Rappels sur PHP
- Créer des requêtes HTTP en PHP
- Les sessions en PHP

Java et HTTP

- Développement d'un serveur HTTP simplifié
- Récupération des données envoyées par le client
- Ecriture d'un client HTTP

HTTP et Éléments WEB

Le protocole HTTP

HTTP : HyperText Transfer Protocol

Basé sur le modèle client/serveur

Client :

- Émission des requêtes
- Réception et affichage des objets Web
- Exemple : un navigateur Web

Serveur :

- Serveur Web attend des connexions
- Répond aux requêtes

Principe

Mode connecté (TCP) non persistant

Etablissement de connexions entre les clients et le serveur

Plusieurs connexions indépendantes établies en parallèle avec le serveur Web

- autant de connexion que d'objet dans la page (image, applet ...)
- possibilité d'optimisation au niveau du navigateur

Pas de mémorisation des informations entre les connexions : **mode sans état**

- Pas d'exploitation des appels précédents
- Pas de gestion du suivi

Cyril Rabat

https://www.cyril-rabat.fr90 %Rechercher

Cyril RabatCVEnseignementsOutilsDiversContactPlan

Cyril Rabat

Home

Je suis actuellement Maître de Conférences à l'Université de Reims Champagne-Ardenne, au sein du département Mathématiques, Informatique et Mécanique. J'ai soutenu ma thèse le 10 décembre 2007, encadrée par Alain Bul, Professeur des universités à l'UFR des Sciences de l'Université de Versailles Saint-Quentin-en-Yvelines et Olivier Flauzac, Professeur des universités à l'U.F.R. Sciences Exactes et Naturelles de l'Université de Reims Champagne-Ardenne. Je fais partie du laboratoire  **CRestIC** au sein de l'équipe CASH. Je travaille dans le domaine des systèmes distribués et en particulier sur les thèmes des grilles informatiques, des systèmes pair-à-pair et des réseaux de capteurs.

Curriculum Vitae

Vous pouvez consulter mon *Curriculum Vitae* dans cette section. Il présente mon parcours universitaire et professionnel, résume mes différentes publications et enseignements.

[Aller »](#)

Enseignements

Cette section comprend les différentes pages associées aux matières que j'enseigne à l'Université de Reims Champagne-Ardenne ou au CNAM Champagne-Ardenne, en formation à distance.

[Aller »](#)

Connexion

[Connexion](#)[Mot de passe perdu](#)

Dernières nouvelles

20/07/2020 Vous pouvez consulter la page "préentrée" pour avoir les informations sur les journées de préentrées en Licence INFO.

16/07/2020 La première phase des inscriptions se termine le 21 juillet. La période pour la deuxième phase est du 19 au 31 août.

25/06/2020 La période pour la phase

InspecteurConsoleDébogueurRéseauÉditeur de stylePerformancesMémoireStockageAccessibilitéApplications

Filtrer les URL

ToutHTMLCSSJSXHRPolicesImagesMédiasWSAutre

☐ Désactiver le cacheAucune limitation de la bande passante

État	Méthode	Domaine	Fichier	Initiateur	Type	Transfert	Taille	0 ms	320 ms
200	GET	www.cyril-rabat.fr	/	document	html	3,15 Ko	9,38 Ko	109 ms	
200	GET	www.cyril-rabat.fr	bootstrap.min.css	stylesheet	css	17,11 Ko	98,12 Ko	25 ms	
200	GET	www.cyril-rabat.fr	bootstrap-theme.min.css	stylesheet	css	2,01 Ko	13,04 Ko	22 ms	
200	GET	www.cyril-rabat.fr	theme.css	stylesheet	css	460 o	203 o	22 ms	
200	GET	www.cyril-rabat.fr	mon.css	stylesheet	css	479 o	301 o	22 ms	
200	GET	www.cyril-rabat.fr	jasny-bootstrap.min.css	stylesheet	css	2,72 Ko	13,67 Ko	22 ms	
200	GET	www.cyril-rabat.fr	bootstrap-datetimepicker.css	stylesheet	css	1,64 Ko	8,80 Ko	22 ms	
200	GET	www.cyril-rabat.fr	jquery.min.js	script	js	32,90 Ko	94,12 Ko	30 ms	
200	GET	www.cyril-rabat.fr	bootstrap.min.js	script	js	7,75 Ko	28,48 Ko	24 ms	
200	GET	www.cyril-rabat.fr	jasny-bootstrap.min.js	script	js	5,07 Ko	16,39 Ko	24 ms	
200	GET	www.cyril-rabat.fr	html.png	img	png	1,43 Ko	1,18 Ko	21 ms	
200	GET	www.cyril-rabat.fr	logo.jpg	img	jpeg	96,42 Ko	96,17 Ko	29 ms	
200	GET	www.cyril-rabat.fr	favicon.ico	FaviconLoader.jsm:179...	x-icon	mis en cache	1,37 Ko		0 ms

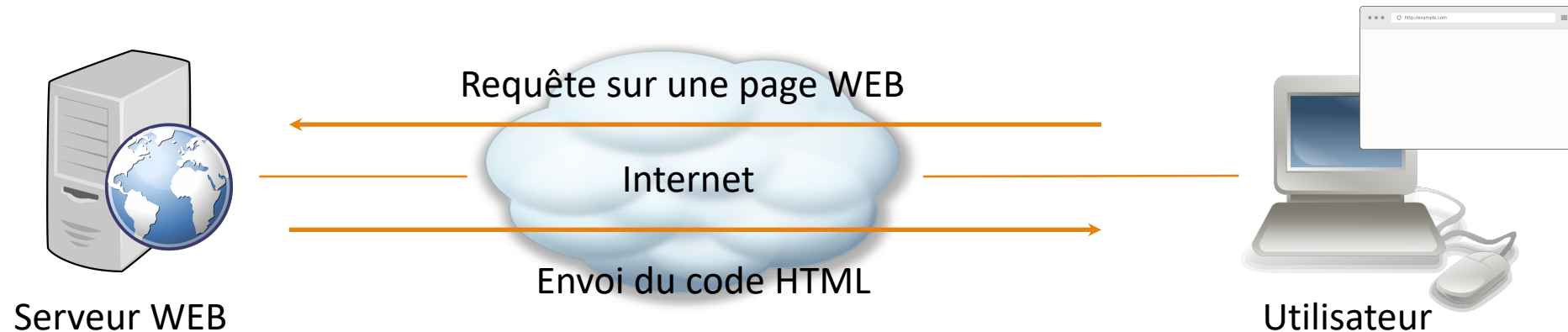
13 requêtes381,21 Ko / 171,12 Ko transférésTerminé en : 379 msDOMContentLoaded: 259 msload: 296 ms

Fonctionnement de HTTP

- HTTP est encapsulé dans TCP
 - Les éléments de HTTP sont les données transmises en TCP
 - Le serveur Web écoute sur le port 80

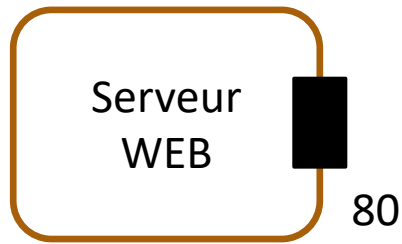
Une fois la connexion établie, échanges de messages HTTP

- Messages au format ASCII



Communication HTTP

Etape 1 : écoute du serveur WEB sur port 80



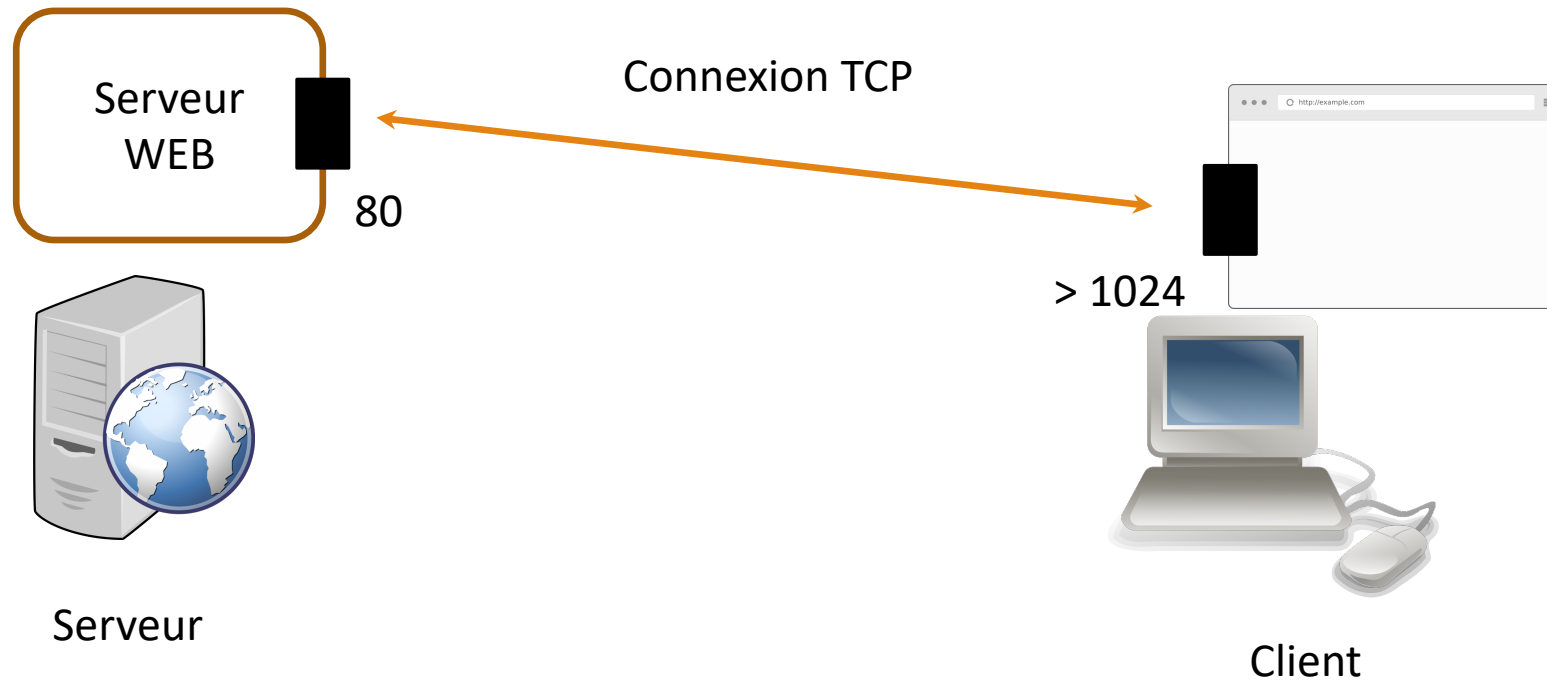
Serveur



Client

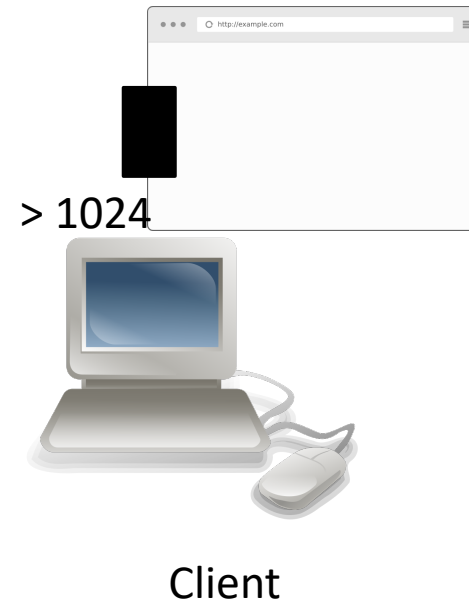
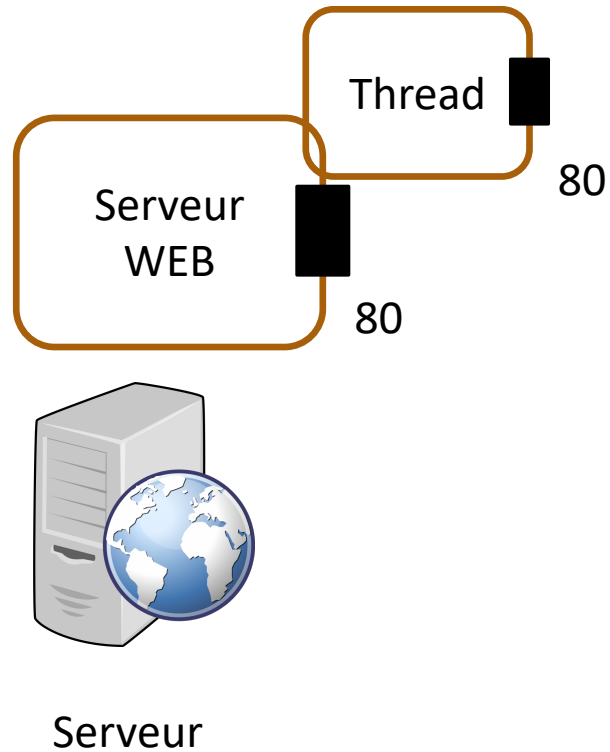
Communication HTTP

Etape 2 : établissement d'une connexion TCP



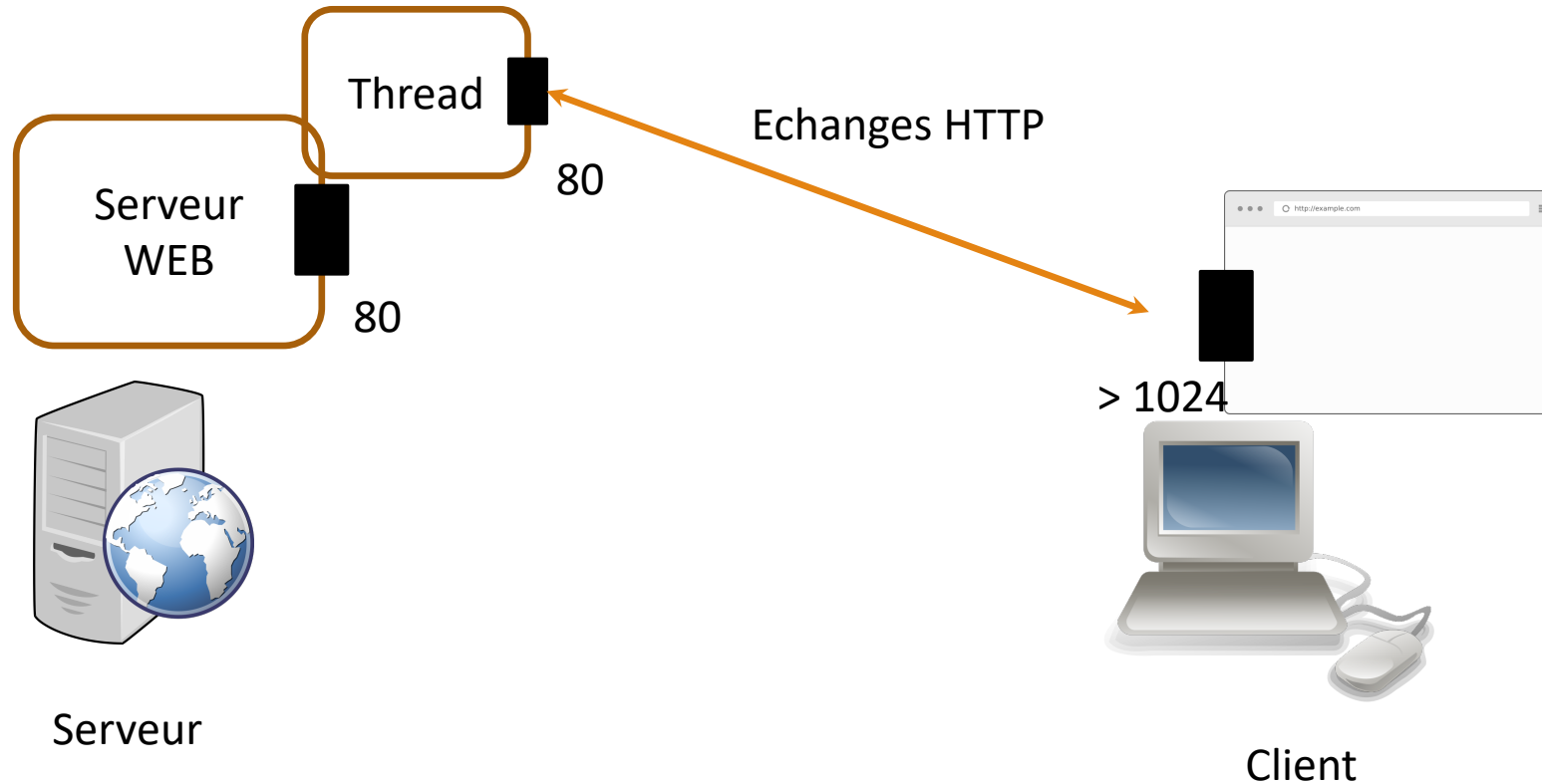
Communication HTTP

Etape 3 : création d'un thread sur le serveur



Communication HTTP

Etape 4 : échange de données



Les messages

Deux types de messages

Mode client / serveur

Connexion à l'initiative du Client

Client → Serveur

- Requête : demande de service

Serveur → Client

- Réponse : émission du résultat du traitement demandé

La requête

Première ligne : commande

Lignes suivantes : lignes d'en-tête

terminée par une ligne vide

Données placées ensuite

```
GET /index.html HTTP/1.0
Accept : text/html
Accept-Language : fr,fr-fr ;q=0.8,en-us
User-Agent : Mozilla/5.0 (Macintosh ; Intel Mac OS X 10.7 ; rv :15.0) Gecko/20100101 Firefox/15.0
\n
```

Types de requêtes

GET : demande d'une ressource

POST : fourniture de données à une ressource

PUT : remplacement ou ajout d'une ressource (non activé)

HEAD : demande d'informations sur une ressource

DELETE : suppression d'une ressource (non activé)

OPTIONS, CONNECT, TRACE, PATCH

La réponse

Première ligne : état

Lignes suivantes : lignes d'en-tête

Terminée par une ligne vide

Données placées ensuite

```
HTTP/1.x 200 OK
Connection : close
Date : Wed, 03 Feb 2010 06 :09 :02 GMT Server : Apache
Content-Length : 6821
Content-Type : text/html

données données données données données
```

Codes réponse

1XX : en cours de traitement

2XX : Succès

- 200 OK

3XX : Redirection

4XX : erreurs côté client

- 404 not found
- 403 forbidden

5XX : erreurs côté serveur

- 500 internal server error

Format des données

MIME : Multimedia Mail Extension

Ensemble de types pour déchiffrer les données

Définition hiérarchique : type/sous type

Différents types :

- Texte (plain, html)
- Image (jpeg, fig), audio (basic, 32kadpcm), vidéo (mpeg, quicktime)
- Application (msword, octet-stream)

Lignes d'en-tête en HTTP :

- Content-Type: image/jpeg → type MIME
- Content-Length: 66 → taille en octets
- Content-Transfer-Encoding: base64 → encodage des données

Echange de données

Les formulaires

POST ou GET ?

Méthode de transmission d'informations

POST

- transport des données dans le corps de la requête
- Pas de cryptage du contenu !

GET

- transport des données dans l'URL

Limite de la taille des données

- Définie par le navigateur
- Définie par le serveur

Un formulaire

Envoi d'informations au serveur Web

Ensemble de champs/éléments :

- Champs texte, radiobutton, checkbox, listes déroulantes...

Utilisation de l'élément <form>

Différents attributs :

- action : nom du script appelé lors de la validation
- method : GET/POST ⇒ méthode d'envoi des données (cf HTTP)

```
<form action="log.php" method="post">  
    Login : <input name="login" type="text"/>  
    Mot de passe : <input name="motDePasse" type="password"/>  
    <button type="submit">Connexion</button>  
</form>
```

Validation d'un formulaire

Envoi des données

- Lors de la validation : données du formulaire envoyées
 - Contenues dans la requête HTTP
- Format de l'envoi suivant la méthode :
 - GET : données ajoutées à l'URL
 - Séparation de l'URL et des données par un " ?"
 - Couples "nom=données" séparés par un "&"
 - POST : données ajoutées dans les données du message HTTP

Validation avec la méthode GET

La requête HTTP

```
GET /formulaires/log.php?login=toto&motDePasse=toto HTTP/1.1
Host: www.cyril-rabat.fr
Connection: keep-alive
User-Agent: Mozilla/5.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Referer: http://www.cyril-rabat.fr/formulaires/
Accept-Encoding: gzip,deflate,sdch
Accept-Language: fr-FR,fr;q=0.8,en-US;q=0.6,en;q=0.4
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.3
Cookie: PHPSESSID=????
```

Validation avec la méthode POST

La requête HTTP

```
POST /formulaires/log.php HTTP/1.1
Host: www.cyril-rabat.fr
Connection: keep-alive
Content-Length: 26
Cache-Control: max-age=0
Origin: http://www.cyril-rabat.fr
User-Agent: Mozilla/5.0
Content-Type: application/x-www-form-urlencoded
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Referer: http://www.cyril-rabat.fr/formulaires/
Accept-Encoding: gzip,deflate,sdch
Accept-Language: fr-FR,fr;q=0.8,en-US;q=0.6,en;q=0.4
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.3
Cookie: PHPSESSID=???

login=toto&motDePasse=toto
```

Sessions et cookies

Suivi de navigation

Le protocole HTTP est sans état

- Pas de suivi entre les scripts

Le suivi de navigation :

- Conservation des données du client tout au long des échanges

Deux solutions :

- Transférer les données à chaque échange :
 - Formulaire avec champs cachés
 - Cookie
- Stocker les données sur le serveur et l'associer à une clé :
 - Session

Un champ caché

Dans un formulaire, possibilité de créer un champ caché :

- `<hidden type="hidden" name="login" value="toto"/>`

Champ lisible par l'utilisateur (afficher la source) :

- Possibilité de modifier cette valeur sans contrôle (injection)

Obligation de passer d'une page à une autre via un formulaire :

- Que faire des liens ?

Le cookie

Élément du protocole HTTP

Enregistrement des données du côté client :

- Associations clé/valeur (chaînes de caractères)
- Stockage sous forme de fichiers sur le client

Données envoyées dans chaque requête HTTP :

- Nom et valeur du cookie spécifiés dans l'en-tête

Information limitée au site courant :

- Pas possible de récupérer un cookie d'un autre site

Limite des cookies

Suivant le navigateur :

- La taille et le nombre de cookies sont limités

Toutes les données doivent être transférées à chaque requête

Stockage sous forme de chaîne de caractères :

- Utilisation de la sérialisation
- Séparation des données par un délimiteur
- Réalisation d'un *parsing*

Remarque

- Des langages tels que PHP facilitent la récupération des données

Les sessions

Génération d'une clé sur le serveur, envoyée au client

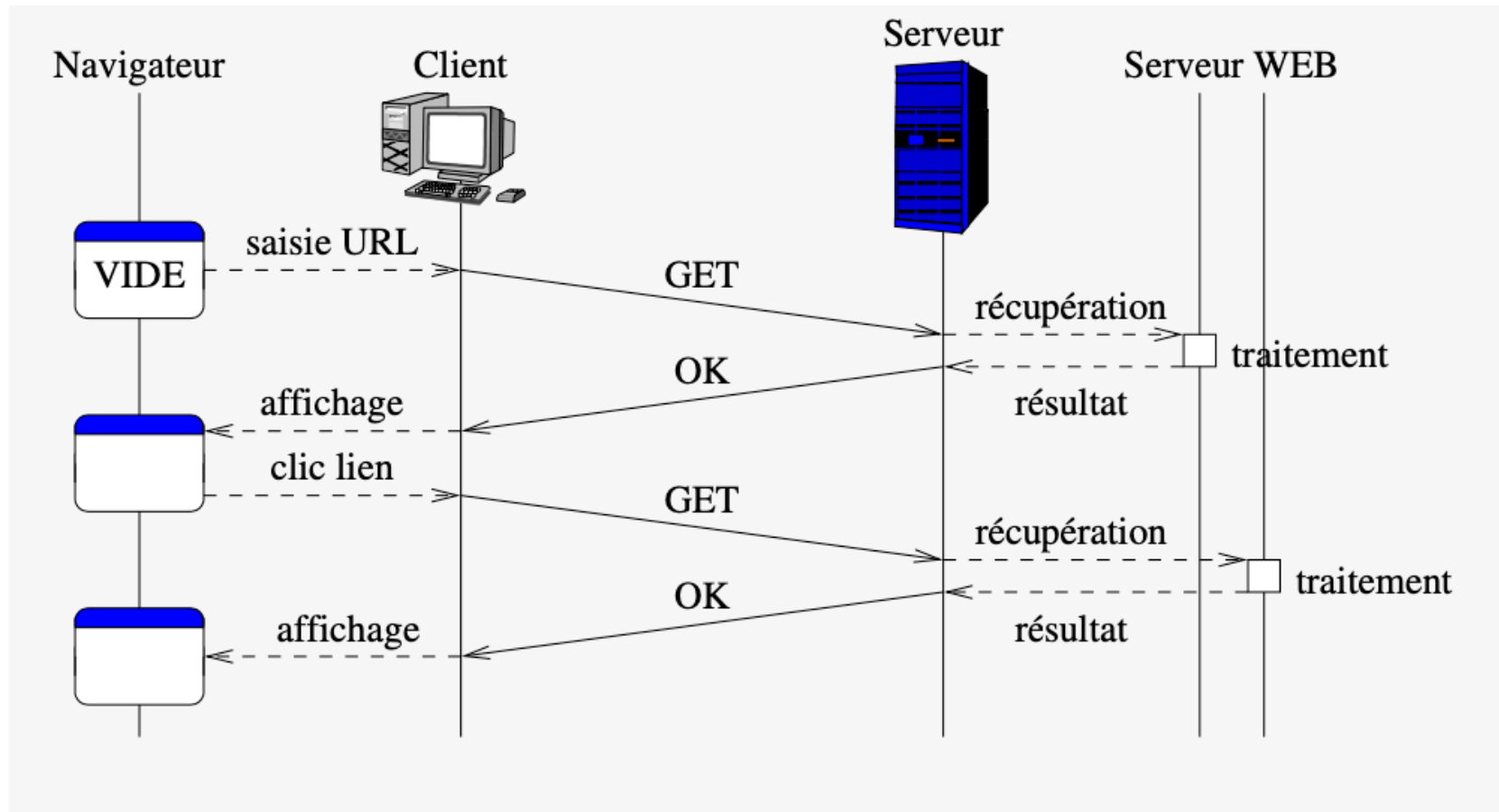
À chaque échange, envoi uniquement de la clé par le client

- Le client est donc "reconnu" par le serveur

Sur le serveur, une session est créée :

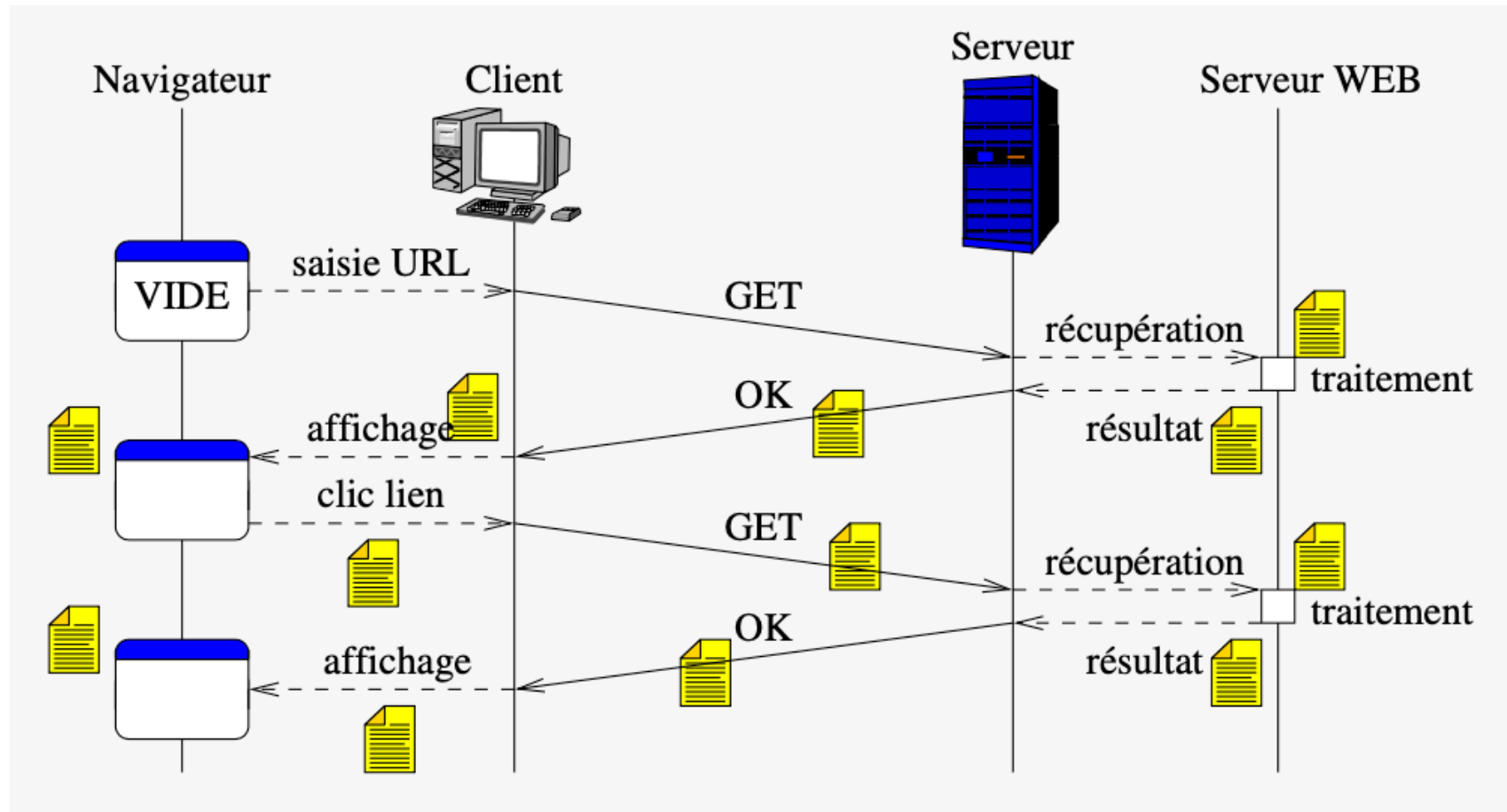
- La clé est associée à la session de ce client
- Possibilité d'associer un jeu de données stocké sur le serveur
- Tout type de données possible :
 - Dépend uniquement du langage !

Série de requêtes / réponses HTTP

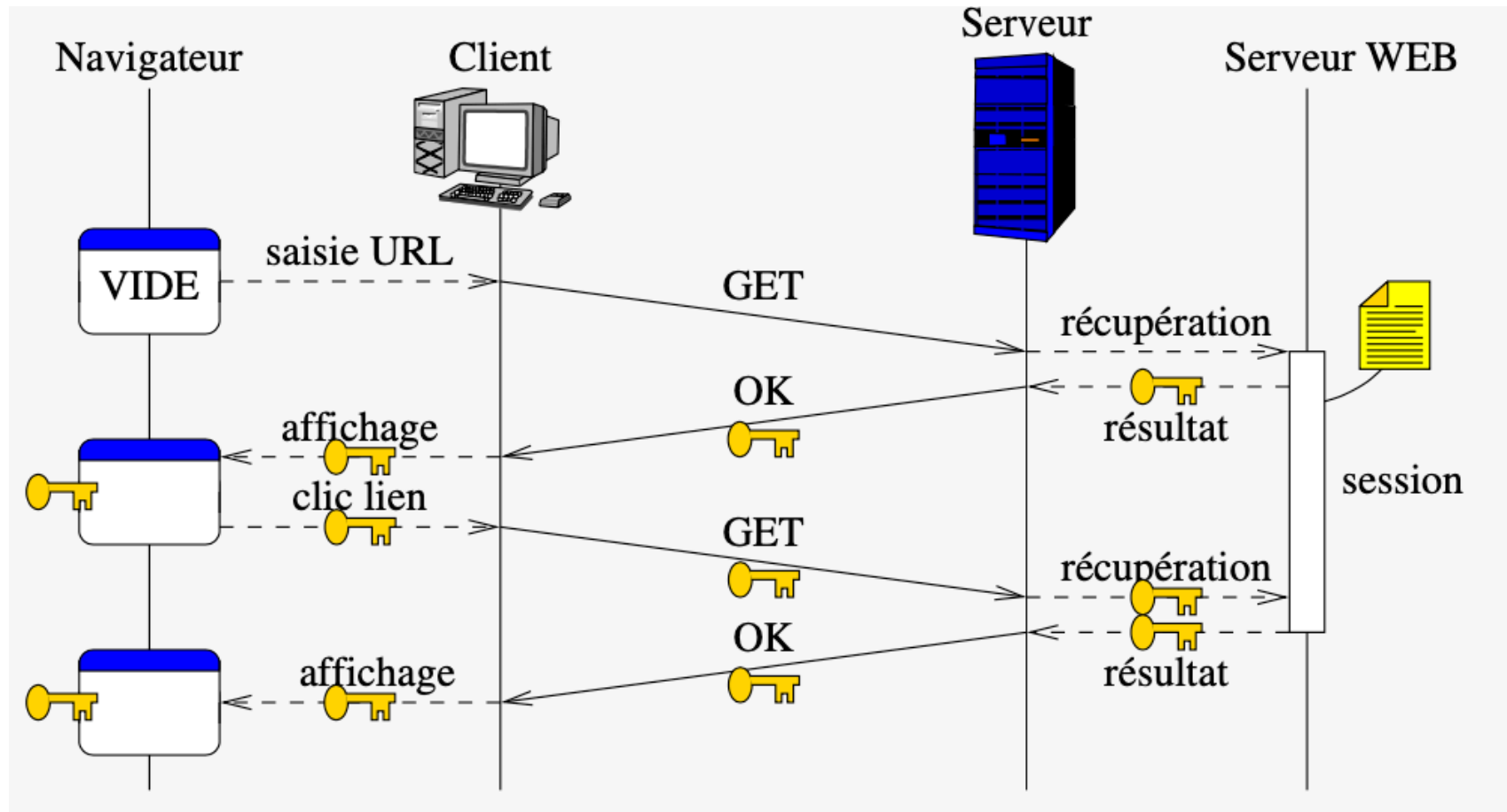


Série de requêtes / réponses HTTP

Avec des cookies



Série de requêtes / réponses HTTP Avec des Session



PHP et HTTP

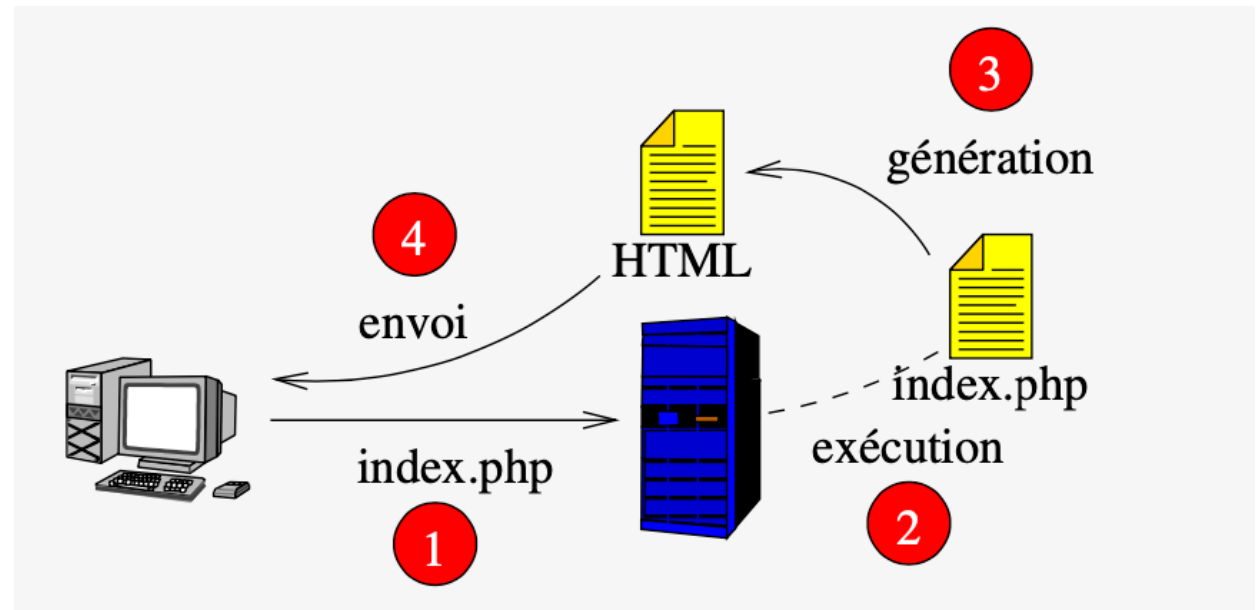
PHP et HTML

Code inclus en tout point du HTML

Utilisation des balises `<?php` et `?>` :

- Le code PHP est interprété par le serveur
- Remplacement du code par le résultat (affiché via echo)

Pas de PHP sur le client!



Prétraitement des requêtes http

Les requêtes HTTP sont mises en forme par le moteur PHP :

- Les données sont facilement accessibles dans le script

Utilisation de variables globales contenant les informations

Plusieurs tableaux associatifs :

- `_GET` : données passées en GET
- `_POST` : données passées en POST
- `_COOKIES` : les cookies

Attention à la configuration de PHP !!!

Si l'option `register_globals` est à ON, une variable est créée pour chaque donnée ! À éviter...

Création de requêtes HTTP

La création de la réponse HTTP est automatisée

Dès que l'on écrit (avec un `echo`, par exemple) :

- La réponse HTTP est formée
- le contenu est considéré comme du HTML

Création automatique mais contrôlée par le programmeur

- Possibilité de modifier les champs HTTP
- Contenu autre que HTML possible !

Attention cependant à l'ordre

Attention lors de la création d'un cookie

- La création d'un cookie doit être réalisée au début
- Dans le cas contraire : erreur (l'en-tête HTTP a déjà été créée !)

Le cas de la redirection

Possibilité de rediriger vers un autre script/page avec header

Que se passe-t-il ?

- La redirection est réalisée par le moteur PHP ?
- Par le serveur Web ?
- Par le client ?

Exemple : code HTML

```
<html>
  <head>
    <title>Exemple de redirection</title> </head>
  <body>
    <h1>Exemple de redirection</h1>
    <a href="script1.php">Pas chiche de me cliquer dessus !</a>
  </body>
</html>
```

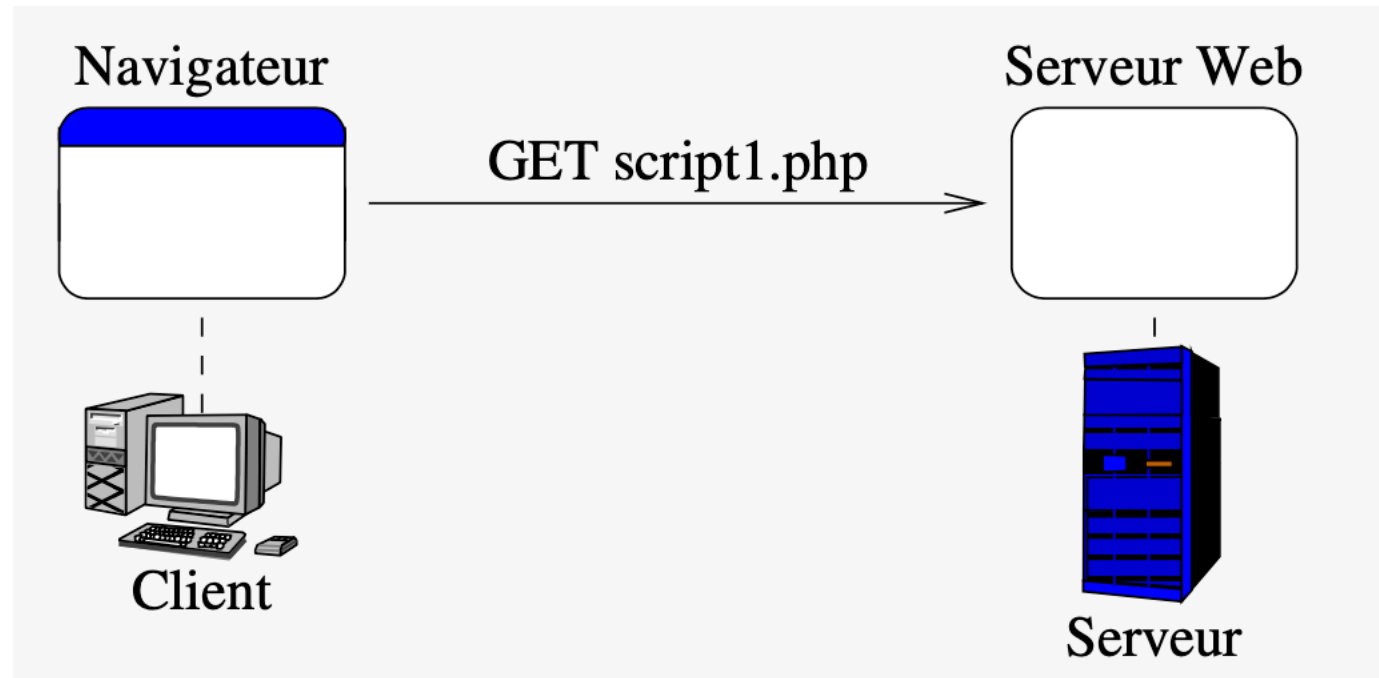
Exemple : scripts PHP

```
<?php
    header("Location : script2.php") ;
?>
```

```
<?php
    echo <<<END
    <html>
        <head>
            <title>Script 2</title>
        </head>
        <body>
            <h1>Script 2</h1>
            <p>Vous avez été redirigé à l'insu de votre plein gré !!!</p>
        </body>
    </html> END ;
?>
```

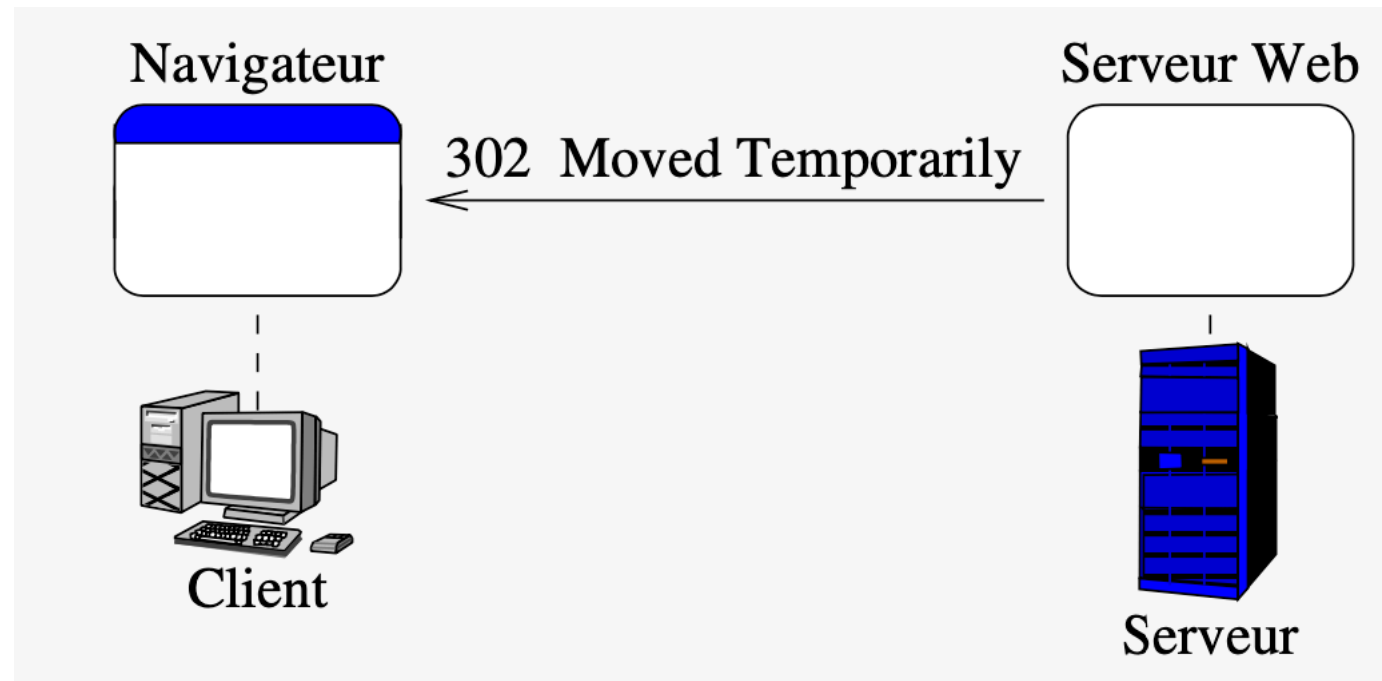
Exécution

Lorsque le lien est cliqué, un GET sur script1.php est exécuté



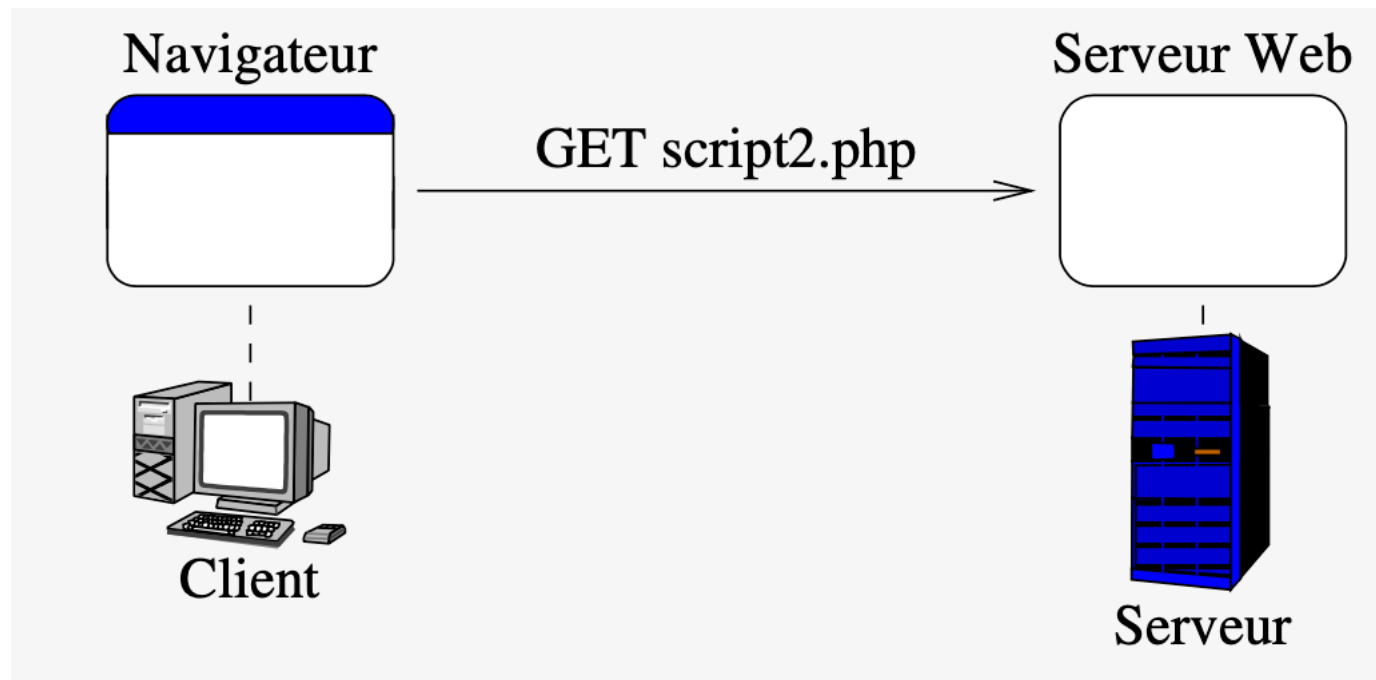
Exécution

Le script PHP fait une redirection :
une requête HTTP de type 302 est envoyée



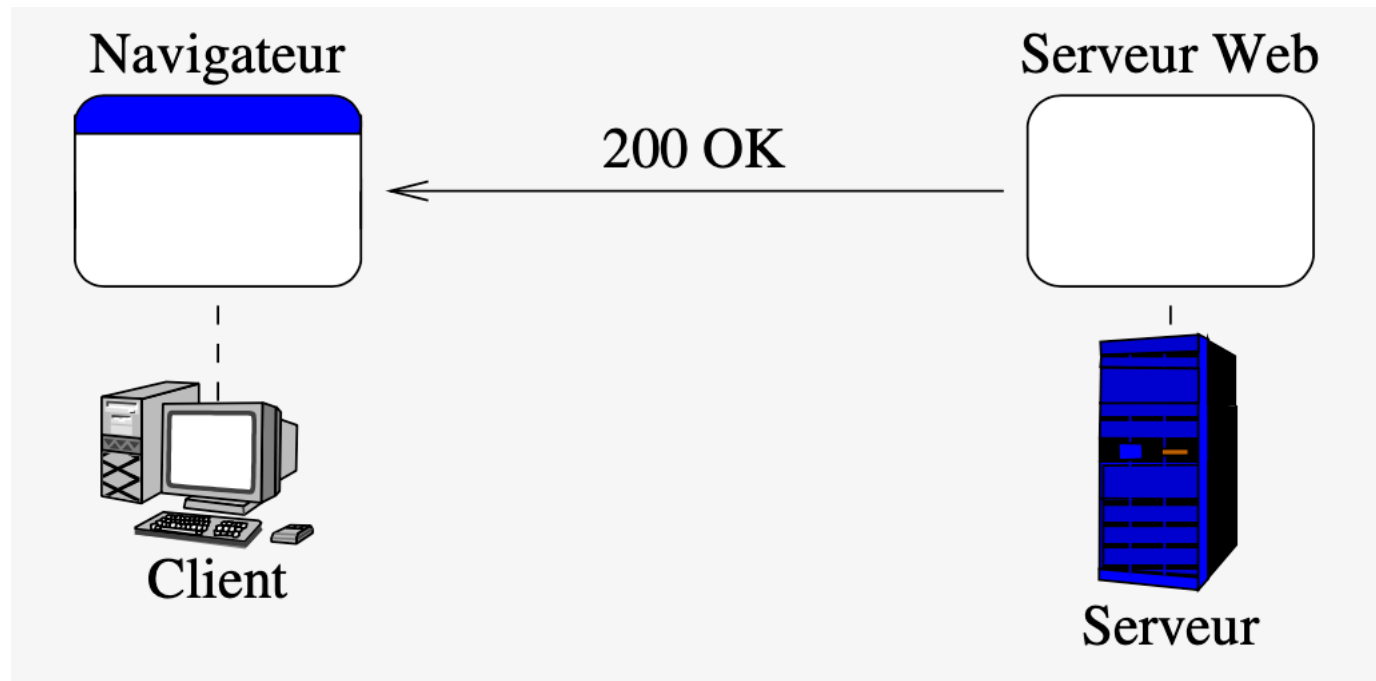
Exécution

Le client peut réémettre une nouvelle requête vers script2.php



Exécution

Le code HTML généré par le script PHP est envoyé dans un message HTTP



En-têtes et PHP

Possibilité de former une requête manuellement

- Utilisation de la commande header
- Permet de spécifier chaque ligne de l'en-tête HTTP

Génération autre que du HTML : images, PDF, XML, etc.

- À suivre dans un prochain cours

```
header("Pragma : public") ;
header("Expire : 0") ;
header("Cache-Control : must-revalidate, post-check=0, pre-check=0") ;
header("Cache-Control : private",false) ;
header("Content-Description : File Transfer") ;
header("Content-Type : application/pdf") ;
header("Content-Disposition : attachment ; filename=\"toto.pdf\"") ;
header("Content-Transfer-Encoding : binary") ;
header("Content-Length : " . filesize("toto.pdf")) ;
readfile("toto.pdf") ;
```

Sessions en PHP

Les sessions peuvent être gérées simplement en PHP

La fonction `session_start()` permet de créer une nouvelle session

Le nom et la clé de la session :

- fonction `session_name()` : nom de la session
 - Généralement PHPSESSID
- Fonction `session_id()` : la clé (identifiant) de la session

Pour enregistrer des données : tableau `$_SESSION['...']`

Remarque sur les objets en PHP

Lorsque des objets sont stockés dans la session, ils sont sérialisés

Configuration de PHP $\Rightarrow$ sérialisation automatique ou non !

Rappel sur les sessions

Permet de "suivre" un client :

1. Création de la session sur le serveur :
 - Génération automatique d'une clé unique
2. Envoi de cette clé au client
3. À chaque requête, le client envoie sa clé
4. À partir de la clé, le serveur récupère les données associées au client
 - Comment passer la clé au client ?

Gestion automatique en PHP

Dépend

- De la configuration du serveur
- De la configuration du client (le navigateur Web)

Fonctionnement par défaut

- La clé est envoyée sous la forme d'un cookie de session
- L'envoi de l'identifiant de session est donc automatique :
 - Transparence totale pour le programmeur
- Rappel : un cookie de session est détruit à la fermeture du navigateur (ou explicitement)

Problème

Le client peut bloquer les cookies !!!

Contourner les cookies

Solution manuelle

- Obligation de passer le nom de la session et la clé par GET
- Tous les liens doivent être modifiés comme suit :
 - `echo "<a href=\"index.php?\".session_name().\"=\".session_id().\"\"> Lien </a>";`

Solution automatique

- Par défaut, PHP détecte que le client bloque les cookies
- Le moteur PHP transforme automatiquement tous les liens

Manipulation à faire

- Créez un script PHP qui crée une session
- Analysez les requêtes HTTP échangées + le code HTML généré
- Idem mais en bloquant les cookies sur le navigateur

Java et HTTP

La classe HTTPServer

Depuis Java 6, serveur Web intégré dans le JDK :

- HttpServer

Contient une API permettant de créer des serveurs embarqués

Création d'un serveur :

1. Instanciation : numéro de port/adresse
2. Association de contextes :
 - Mapping avec les URL
3. Ajout de filtres :
 - Chaînage de traitements automatisés
4. Choix d'un moteur d'exécution
5. Démarrage

Notion de contexte

Mapping entre un URL et un contexte

Le contexte géré par un Handler :

- Implémentation de l'interface Handler

Méthode principale handle appelée :

```
public void handle(HttpExchange t) throws IOException
```

L'objet `HttpExchange` représente l'échange HTTP :

- Contient la requête...
- ...et la réponse (à faire !)

Exemple

Un serveur simple

```
public class ServeurSimple {  
    public static void main(String[] args) {  
        HttpServer serveur = null ;  
        try {  
            HttpServer.create(new InetSocketAddress(8080), 0) ;  
        } catch(IOException e) {  
            System.err.println("Erreur lors de la creation du serveur " + e) ;  
            System.exit(-1) ;  
        }  
        serveur.createContext("/index.html", new AccueilHandler()) ;  
        serveur.setExecutor(null) ;  
        serveur.start() ;  
    }  
}
```

Exemple

Le Handler

```
class AccueilHandler implements HttpHandler {
    public void handle(HttpExchange t) {
        String reponse = "<h1>Bienvenue</h1>" ;
        try {
            t.sendResponseHeaders(200, reponse.length()) ;
            OutputStream os = t.getResponseBody() ;
            os.write(reponse.getBytes()) ;
            os.close() ;
        } catch(IOException e) {
            System.err.println("Erreur lors de l'envoi de la reponse " + e) ;
            System.exit(-1) ;
        }
    }
}
```

Exécution

Une fois le serveur lancé :

- Attente de connexions de clients
- Ici : port 8080 (attention aux ports < 1024 en salles de TP)

À partir d'un navigateur :

`http://localhost:8080/index.html`

Remarque

Le contexte est `index.html` ce qui permet de masquer le fonctionnement à l'utilisateur

Récupération des données

Les données envoyées en GET :

- Rappel : envoyées dans l'URL
- Objet `HttpExchange` : méthode `getRequestURI`

Les données envoyées en POST :

- Rappel : envoyées dans la partie données du message HTTP
- Objet `HttpExchange` : méthode `getRequestBody`

Récupération en GET

La classe AccueilHandler

```
...
public void handle(HttpExchange t) throws IOException {
    String reponse = "<h1>Bienvenue</h1>" ;
    URI requestedUri = t.getRequestURI() ;
    String query = requestedUri.getRawQuery() ;
    reponse += "<p>Données en GET : " ;
    if(query == null){
        reponse += "<b>Aucune</b></p>" ;
    }else{
        reponse += "<b>" + query + "</b></p>" ;
    }
    t.sendResponseHeaders(200, reponse.length()) ;
    OutputStream os = t.getResponseBody() ;
    os.write(reponse.getBytes()) ;
    os.close() ;
}
...
```


Récupération en POST

La classe AccueilHandler

```
...
public void handle(HttpExchange t) throws IOException {
    String reponse = "<h1>Bienvenue</h1>" ;
    BufferedReader br = new BufferedReader(
        new InputStreamReader( t.getRequestBody(),"utf-8")) ;
    query = br.readLine() ;
    reponse += "<p>Données en POST : " ;
    if(query == null){
        reponse += "<b>Aucune</b></p>" ;
    }else{
        reponse += "<b>" + query + "</b></p>" ;
    }
    t.sendResponseHeaders(200, reponse.length()) ;
    OutputStream os = t.getResponseBody() ;
    os.write(reponse.getBytes()) ;
    os.close() ;
}
...
```

Analyse de données

Récupération de l'intégralité des données :

- Exemple : nom=Rabat&prenom=Cyril

Nécessité d'analyser la chaîne :

- Utilisation des expressions régulières
- Rappel : séparation des données avec '&' et '='

Solution proposée : les filtres

- Réalisation de prétraitements avant l'appel du handler
- Possibilité de chaîner les filtres
- Possibilité d'ajouter des propriétés à `HttpExchange` accessibles ensuite depuis tous les handler

Principe général d'un client

Les étapes :

1. Connexion au serveur
2. Envoi d'une requête HTTP
3. Lecture de la réponse
4. Retour à 2 si nécessaire
5. Déconnexion du serveur

Plusieurs possibilités en Java :

- En bas niveau : utilisation d'une socket (en mode TCP)
- En haut niveau : classes URL et URLConnection

Utilisation de Sockets TCP

Établissement d'une connexion avec le serveur

Envoi de la requête :

- Construction manuelle de la requête
- Uniquement la première ligne obligatoire

Lecture de la réponse :

- Lecture de l'en-tête
- Lecture du corps si nécessaire

À savoir :

- Le corps et l'en-tête sont séparés par une ligne vide
- Chaque ligne terminée par '`\r\n`'
- Attention : le corps est constitué d'une suite d'octets

Code client HTTP

```
Socket socket = null ;
try {
    socket = new Socket("localhost", 8080) ;
} catch(UnknownHostException e) {
    System.err.println("Erreur sur l'hôte " + e) ;
    System.exit(-1) ;
} catch(IOException e) {
    System.err.println("Creation socket impossible " + e) ;
    System.exit(-1) ;
}
```

Code de la classe client

```
// Association d'un flux d'entrée et de sortie

BufferedReader input = null ;
PrintWriter output = null ;
try {
    input = new BufferedReader( new InputStreamReader(socket.getInputStream()) ) ;
    output = new PrintWriter(new BufferedWriter(
        new OutputStreamWriter(socket.getOutputStream()), true) ;
} catch(IOException e) {
    System.err.println("Association des flux impossible" + e) ;
    System.exit(-1) ;
}
```

Code de la classe du client

```
String message = "GET /index.html HTTP/1.1\r\n" ;
output.println(message) ;
try {
    String tmp = "" ;
    while(((tmp = input.readLine()) != null) && (tmp.length() != 0))
        System.out.println(tmp) ;
    if(tmp != null) {
        char[] buffer = new char[1024] ;
        int nbLus = 0 ;
        while(((nbLus = input.read(buffer, 0, buffer.length)) >= 0) && (input.ready()))
            System.out.print(buffer) ;
        if(nbLus > 0) {
            System.out.println(new String(buffer, 0, nbLus));
        }else{
            System.out.println() ;
        }
    }
} catch(IOException e) {
    System.err.println("Erreur lors de la lecture " + e) ;
    System.exit(-1) ;
}...
```

Utilisation de URL et URL Connection

Classe URL :

- Crée un URL
- Permet d'établir une connexion : retourne un URLConnection

Interface URLConnection :

- Correspond à la connexion
- Permet de récupérer les en-têtes
- Permet d'envoyer et de lire des données

Un client simple

```
public static void main(String[] args) {
    URL url = null;
    try {
        url = new URL("http ://localhost :8080/") ;
    } catch (MalformedURLException e) {
        System.err.println(e) ;
        System.exit(-1) ;
    }
    URLConnection connexion = null ;
    try {
        connexion = url.openConnection() ;
        connexion.setDoOutput(false) ;
    } catch (IOException e) {
        System.err.println(e) ;
        System.exit(-1) ;
    } ...
}
```

ClientHTTP

```
String resultat = "" ;
try {
    BufferedReader reader = new BufferedReader(
        new InputStreamReader(connexion.getInputStream())) ;
    String tmp ;
    while((tmp = reader.readLine()) != null)
        resultat += tmp ; reader.close() ;
} catch(Exception e) {
    System.err.println(e) ;
    System.exit(-1) ;
}
System.out.println(resultat) ;
}
```

Envoi de données au serveur

En GET : directement dans l'URL !

- Attention à l'encodage !

En POST :

- Récupérer le flux de sortie de la connexion
- Envoi des données en les encodant !

Pour l'encodage : classe URLEncoder, méthode encode

Exemple d'envoi des données

```
URLConnection connexion = null ;
try {
    connexion = url.openConnection() ;
    connexion.setDoOutput(true) ;
} catch(IOException e) {
    System.err.println(e) ;
    System.exit(-1) ;
}
String donnees = "" ;
try {
    donnees += URLEncoder.encode("nom", "UTF-8") + "=" +
        URLEncoder.encode("Cyril", "UTF-8") ;
} catch(UnsupportedEncodingException e) {
    System.err.println(e) ;
    System.exit(-1) ;
}...
```

```
...
OutputStreamWriter writer = null ;
try {
    writer = new OutputStreamWriter( connexion.getOutputStream() ) ;
    writer.write(donnees) ;
    writer.flush() ;
    writer.close() ;
} catch(IOException e) {
    System.err.println(e) ;
    System.exit(-1) ;
} ...
```